MCA Semester - IV

Project - Final Report

A study on "NovaBank: A Secure Full-Stack Digital Banking Web Application with Rule-Based Fraud Detection and Financial Health Analytics"

A Project submitted to Jain Online (Deemed-to-be University)

In partial fulfillment of the requirements for the award of:

Master of Computer Applications

Submitted by:
Chethan BV
USN: 241VMTR01537
Elective: Full Stack Development

Under the guidance of:
[Faculty Name - JAIN Online]
(Faculty - JAIN Online)

Jain Online (Deemed-to-be University)
Bangalore
Date of Submission: 08/08/2026

DECLARATION

I, Chethan BV, hereby declare that the Project Report titled "NovaBank: A Secure Full-Stack Digital Banking Web Application with Rule-Based Fraud Detection and Financial Health Analytics" has been prepared by me under the guidance of [Faculty Name - JAIN Online]. I declare that this project work is towards the partial fulfilment of the University Regulations for the award of the degree of Master of Computer Applications by Jain University, Bengaluru. I have undertaken this project for one semester and I further declare that the work documented in this report is based on the original study undertaken by me and has not been submitted for the award of any other degree or diploma from any university or institution.

Place: _____

Date: 08/08/2026

Name of the Student: Chethan BV
USN: 241VMTR01537

ACKNOWLEDGEMENT

I express my sincere gratitude to Jain Online (Deemed-to-be University) for providing the academic framework and learning environment required to complete this capstone project. The Master of Computer Applications programme has provided valuable exposure to frontend engineering, backend architecture, database design, software security, and project execution.

I am deeply thankful to [Faculty Name - JAIN Online] for the guidance, feedback, and encouragement provided throughout the development of this project. The support received during the design, implementation, debugging, and documentation phases was instrumental in shaping the project into a complete and presentable digital banking solution.

I also acknowledge the contribution of the course structure, learning resources, and practical assignments that helped build the technical foundation needed to implement NovaBank. Finally, I would like to thank my family and well-wishers for their support and motivation during the successful completion of this capstone project.

EXECUTIVE SUMMARY

NovaBank is a production-style full-stack banking web application created as a capstone project in Full Stack Development. The system addresses the academic and practical challenge of building a secure digital banking platform that supports customer and administrator workflows in a single cohesive solution. The application allows customers to register, log in, manage their personal profile, maintain beneficiaries, transfer funds, review transaction history, view fraud analysis, and study their financial health score. Admin users can monitor customers, inspect transactions, view fraud logs, and access a consolidated operational dashboard.

The application is implemented using React, Tailwind CSS, Axios, and React Router on the front end, and Spring Boot, Spring Security, JWT authentication, Spring Data JPA, Maven, and MySQL on the back end. The backend follows a layered architecture with packages for controllers, services, repositories, DTOs, entities, security, configuration, and exception handling. The data model includes users, roles, accounts, beneficiaries, transactions, and fraud logs. Sensitive data is protected through BCrypt password hashing, stateless JWT-based security, and role-based authorization rules.

A notable feature of the project is its rule-based fraud detection mechanism, which evaluates transfers using practical indicators such as large transfer amount, multiple rapid transactions, and transfers made to recently added beneficiaries. In addition, a financial insight module generates a health score, monthly savings rate, and smart suggestions by examining credit and debit behaviour. These features transform the application from a simple CRUD-based banking demo into a more analytical and decision-support oriented system.

The final project demonstrates that the original capstone goals have been substantially achieved. Core banking modules are integrated end to end, the frontend has been redesigned into an enterprise-style banking interface, and the backend supports transactional consistency for money transfer operations. This report documents the system architecture, technologies used, frontend and backend implementation details, database design, security model, project management approach, achieved results, and overall evaluation of the completed solution.

TABLE OF CONTENTS

1. Introduction

Digital banking applications have become an essential part of modern financial systems because customers increasingly expect secure, responsive, and always-available online services. Traditional branch-first models no longer satisfy user expectations for instant balance inquiry, online fund transfer, beneficiary management, and fraud visibility. As a result, the development of secure web-based banking systems has become both an industrial necessity and an academically valuable full-stack engineering exercise.

The NovaBank project was undertaken to design and implement a secure, role-based banking web application using a modern frontend stack and an enterprise-oriented Java backend. The project aims to demonstrate the complete lifecycle of a full-stack product: requirement understanding, database design, API design, authentication and authorization, business logic implementation, frontend integration, user interface refinement, and technical documentation.

The primary problem statement addressed by the project is the need for a digital banking system that goes beyond simple authentication and balance display. A realistic banking application must preserve data consistency during fund transfers, protect sensitive user data, separate customer and administrator privileges, support auditable transaction history, and provide useful analytical insight into transfer risk and spending behaviour. NovaBank was therefore designed as a modular yet production-minded monolithic application that balances academic completeness with real-world design practice.

The project is significant in the field of Full Stack Development because it integrates multiple advanced competencies within one product. It includes secure JWT-based authentication, layered backend architecture, DTO-based API contracts, relational database normalization, transactional service logic, responsive frontend design, and analytics-driven modules such as fraud scoring and financial health evaluation. This makes the project highly suitable for academic presentation, viva discussion, and portfolio demonstration.

2. System Architecture

NovaBank follows a client-server architecture. The React frontend operates as a Single Page Application that communicates with the Spring Boot backend through REST APIs over HTTP. The backend handles authentication, business logic, data access, and security enforcement, while MySQL persists structured banking data. This separation of concerns keeps the user interface independent from the core application logic and allows each layer to evolve more cleanly.

Internally, the backend uses a layered modular architecture. Controllers expose endpoints and accept validated request payloads. Services implement business rules such as profile updates, beneficiary ownership checks, fund transfers, fraud analysis, and financial health calculations. Repositories interact with the database using Spring Data JPA. DTOs shape inbound and outbound payloads to avoid exposing persistence entities directly. Security classes manage JWT validation, authentication setup, and route authorization.

The system architecture is intentionally designed as a modular monolith instead of a microservice system. For a capstone project, this approach keeps deployment simpler and makes the codebase easier to explain during demonstration while still supporting a realistic separation of responsibilities. The frontend redesign further aligns the system with production expectations by using an enterprise-style shell layout, dense operational tables, and task-oriented navigation rather than marketing-style visuals.

Table 1. Major Architecture Layers

Layer	Role in the System
Frontend	React SPA responsible for login, routing, dashboards, forms, and operational UI rendering.
Controller Layer	Receives HTTP requests and delegates processing to service classes.
Service Layer	Executes domain logic such as transfer, fraud scoring, and insight calculation.
Repository Layer	Manages persistence through Spring Data JPA repositories.
Database Layer	Stores normalized banking records in MySQL.

Figure 1. Context Diagram of NovaBank

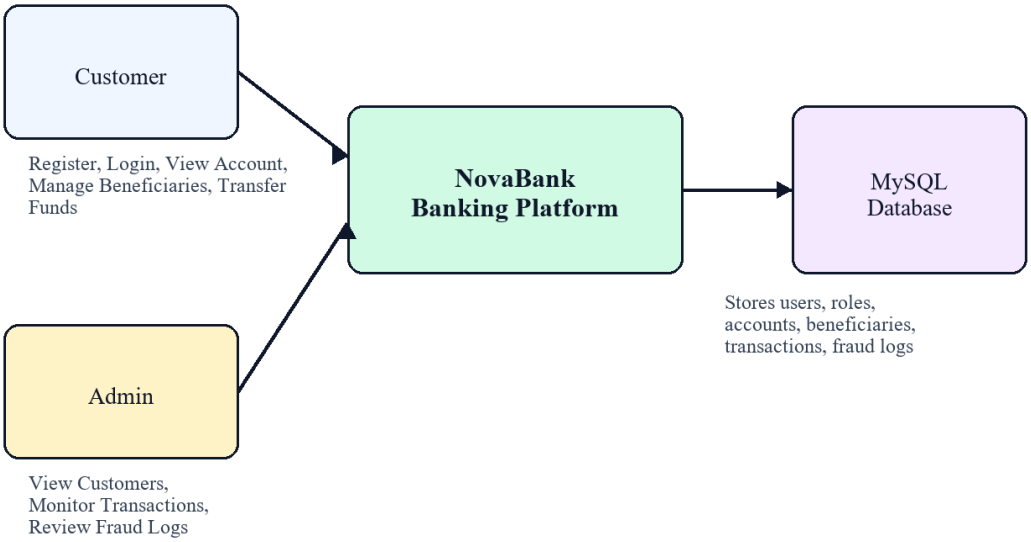

Figure 1. Context Diagram of NovaBank

Figure 2. Layered System Architecture

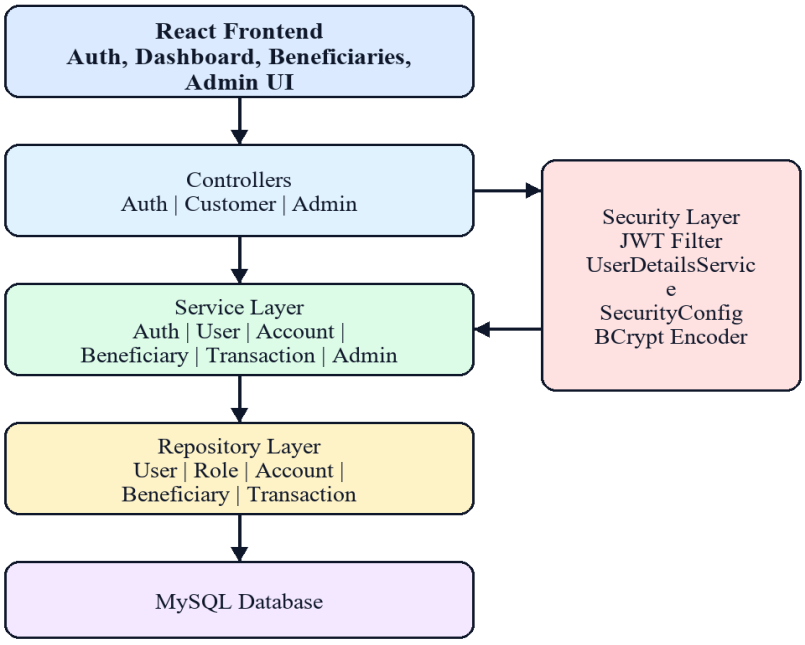

Figure 2. Layered System Architecture

Figure 5. Authentication Sequence Diagram

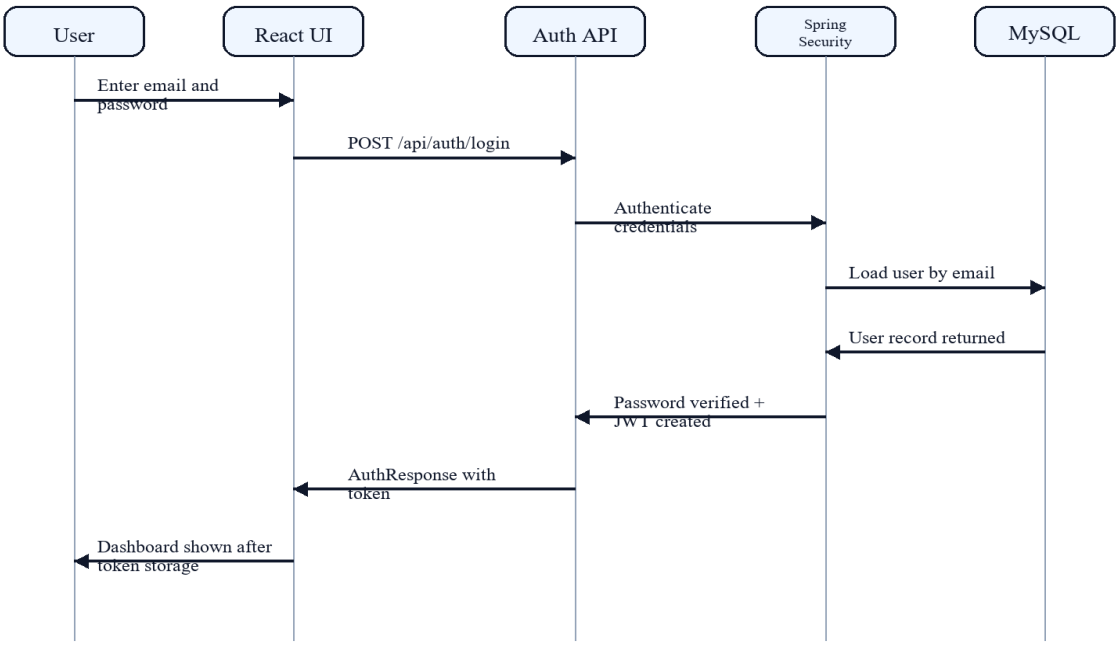

Figure 3. Authentication Sequence Diagram

3. Technologies Used

The project uses a carefully selected technology stack that balances developer productivity with enterprise relevance. React and Vite provide a modern frontend development experience, Tailwind CSS supports rapid yet disciplined UI composition, Axios manages API communication, and React Router handles protected route navigation. On the backend, Java 17 and Spring Boot provide a strong enterprise foundation, Spring Security enforces route protection, and Spring Data JPA accelerates repository development without sacrificing structured relational design.

MySQL was selected as the relational database because the banking domain is transaction-centric and benefits from strong schema definition, referential integrity, and structured query capabilities. Maven is used as the backend build tool due to its maturity and widespread use in Java enterprise environments. Swagger/OpenAPI support assists API inspection and manual verification during development.

Table 2. Technology Stack

Layer	**Technologies / Tools**
Frontend	React 18, Vite, Tailwind CSS, Axios, React Router
Backend	Java 17, Spring Boot 3, Spring Security, JWT, Spring Data JPA, Maven
Database	MySQL
Documentation & API Inspection	Swagger UI via SpringDoc OpenAPI, project report scripts
Build & Packaging	Vite production build and Maven package lifecycle

Table 3. Technology Selection Rationale

Technology	**Reason for Use**
React + Vite	Fast SPA development, routing flexibility, and reusable component model.
Tailwind CSS	Consistent utility-driven styling for enterprise UI composition.
Spring Boot	Mature Java backend framework suitable for enterprise-style layered applications.
Spring Security + JWT	Secure stateless authentication and role-based authorization.
Spring Data JPA	Simplifies repository development over normalized relational entities.

Technology	Reason for Use
MySQL	Reliable relational storage with referential integrity and transactional support.

4. Front-end Development

The frontend is structured around React page components, reusable UI primitives, an authentication context, and a centralized Axios service. Routes are defined in the main application router and protected using a dedicated `ProtectedRoute` component. This ensures that unauthenticated users are redirected away from secured pages, while role constraints continue to be enforced by the backend.

A significant improvement made during the final phase of the project was the redesign of the user interface to resemble a realistic enterprise banking portal. The final interface avoids common AI-generated dashboard clichés such as oversized rounded cards, translucent glass surfaces, loud gradients, floating panels, and hero-style landing sections. Instead, the application now uses compact spacing, sharper information panels, structured navigation, dense tables, operational labels, and conservative blue-grey colour treatment similar to established banks.

The key customer-facing pages include login, registration, account overview, payments and transfers, beneficiary management, transaction history, fraud review, financial insights, and profile maintenance. The admin-facing interface provides an operations dashboard with customer search, transaction visibility, fraud queue inspection, and platform summary metrics. Shared formatting utilities standardize currency display, transaction time display, account masking, and label formatting across pages.

The user experience is designed around clarity and efficiency rather than visual novelty. Tables expose real banking attributes such as account numbers, references, debit and credit columns, narration, status, and timestamps. Forms use compact enterprise input treatment with plain borders, clear labels, and restrained button styling. This design direction makes the project suitable for both academic review and professional portfolio presentation.

User Interface and User Experience Considerations

- Enterprise-style layout with top header, left navigation, dense tables, and operational panels.
- Accessibility-friendly form controls with clear labels, compact spacing, and restrained colour use.
- Protected route handling so unauthenticated users are redirected away from customer or admin screens.
- Shared formatting utilities for consistent currency, dates, and account masking across the interface.

Table 4. Frontend Pages and Their Purpose

Page	Purpose
Login	Authenticates customer or admin users and routes them to the relevant dashboard.
Register	Creates new customer profiles and provisions access credentials.
Account Overview	Displays account summary, balance, service notices, and recent transactions.
Payments & Transfers	Submits money transfer requests using saved beneficiaries.
Beneficiary Management	Creates, updates, and removes reusable transfer beneficiaries.
Transaction History	Provides searchable debit and credit ledger visibility.
Fraud Review	Displays rule-based fraud scores and investigation reasons.
Financial Insights	Shows health score, cash-flow trends, and smart suggestions.
Profile & Contact	Maintains customer name, phone, and registered address.
Operations Dashboard	Allows admins to monitor customers, volumes, and fraud queue entries.

5. Back-end Development

The backend is implemented with Spring Boot using a layered package structure: `controller`, `service`, `repository`, `dto`, `entity`, `config`, `security`, and `exception`. This separation improves maintainability because HTTP concerns, business rules, persistence logic, and security configuration remain clearly isolated. Each controller is concise and delegates logic to the relevant service layer, which is a common enterprise backend practice.

The authentication layer exposes two public endpoints: customer registration and login. All other operational endpoints are protected and role aware. The customer controller provides profile management, account summary retrieval, beneficiary CRUD operations, transaction listing with search support, transfer execution, fraud analytics retrieval, and financial insight retrieval. The admin controller provides dashboard metrics, customer listing, transaction listing, and fraud log access.

One of the most important backend flows is the transfer service. When a transfer request is received, the service loads the sender account and validates beneficiary ownership. The sender and receiver accounts are locked using repository methods intended for update-sensitive access. The service checks that both accounts are active, confirms that sender and receiver are different, validates sufficient balance, updates both balances atomically, saves both ledger records, and stores the fraud evaluation result. The sender receives a debit transaction entry, while the receiver receives a credit transaction entry sharing the same reference number.

The backend also includes a centralized exception strategy and validation annotations on incoming DTOs. This improves API predictability by ensuring that malformed requests, unauthorized access, missing resources, and business rule failures are surfaced consistently. The result is a cleaner and more defendable backend for viva discussion.

Table 5. Backend Package Responsibilities

Package	Responsibility
controller	Defines REST endpoints for authentication, customer, and admin workflows.
service	Implements business logic such as transfers, fraud scoring, and insight calculation.
repository	Performs JPA-based data access and update-sensitive account lookups.
dto	Defines request and response payloads to isolate API contracts from entities.

Package	Responsibility
entity	Represents relational domain models such as account, transaction, and fraud log.
security	Configures Spring Security, JWT filtering, and user-details loading.
config	Holds application-level configuration and demo data seeding logic.
exception	Provides centralized exception classes and a global exception handler.

Key Backend Functional Flows

- Authentication flow creates JWT responses for both customer and admin roles.
- Beneficiary service enforces owner-specific access to payee records.
- Transaction service performs debit and credit ledger writes inside a transactional boundary.
- Admin service aggregates customer, transaction, and fraud data for operations review.

Table 6. Core REST APIs

Method	Endpoint	Purpose
POST	/api/auth/register	Registers a customer and returns authenticated response data.
POST	/api/auth/login	Authenticates a user and returns JWT token with profile details.
GET	/api/customer/profile	Fetches customer profile details.
PUT	/api/customer/profile	Updates profile contact details.
GET	/api/customer/account	Fetches account summary and balance information.
GET	/api/customer/beneficiaries	Lists beneficiaries owned by the logged-in customer.
POST	/api/customer/beneficiaries	Creates a new beneficiary.
PUT	/api/customer/beneficiaries/{id}	Updates an existing beneficiary.
DELETE	/api/customer/beneficiaries/{id}	Removes a beneficiary.
GET	/api/customer/transactions	Lists customer transactions with optional search.
POST	/api/customer/transfer	Executes secured transfer and records sender and receiver ledgers.

Method	Endpoint	Purpose
GET	/api/customer/fraud-analytics	Returns customer fraud analysis history.
GET	/api/customer/financial-insights	Returns health score, trends, and suggestions.
GET	/api/admin/dashboard	Returns aggregate admin dashboard statistics.
GET	/api/admin/customers	Lists customer details without exposing password data.
GET	/api/admin/transactions	Lists all transactions for administrative review.
GET	/api/admin/fraud-logs	Lists fraud log entries for operations monitoring.

6. Database Design

NovaBank uses MySQL as its relational database management system. This choice is appropriate because the banking domain depends on strongly structured data, referential integrity, predictable joins, and transactional consistency. The schema has been normalized across users, roles, accounts, beneficiaries, transactions, and fraud logs to reduce redundancy and simplify rule enforcement.

The `users` table stores identity and contact information, while `roles` and `user_roles` implement normalized authorization mapping. The `accounts` table is separated from `users` to allow account-specific attributes such as account number, balance, currency, and status to evolve independently. The `beneficiaries` table allows reusable payee management for each customer. The `transactions` table acts as an immutable ledger, and the `fraud_logs` table stores rule-based risk analysis linked to transfer activity.

A notable refinement in the final implementation is the structure of the transaction entity. Each transfer can create both a sender-side debit row and a receiver-side credit row under a shared reference number. The entity includes sender account, receiver account, ledger owner account, amount, type, description, beneficiary details, and balance after transaction. This design improves traceability, customer visibility, and administrative auditability.

Table 7. Database Tables and Purpose

Table	Purpose
roles	Stores role definitions such as ROLE_ADMIN and ROLE_CUSTOMER.
users	Stores customer/admin identity, contact details, password hash, and status.
user_roles	Maintains normalized many-to-many mapping between users and roles.
accounts	Stores account number, balance, currency, active flag, and linked user.
beneficiaries	Stores beneficiary nickname, payee identity, bank details, and owner.
transactions	Stores immutable debit/credit ledger records with sender/receiver linkage.
fraud_logs	Stores evaluated risk score, risk level, amount, and reasons for transfers.

Table 8. Normalization and Design Justification

Design Choice	Justification
User and role split	Avoids repeated role text in each user record and supports flexible role assignment.
Account separated from user	Keeps financial attributes independent from identity attributes.
Beneficiary table per customer	Enables reusable payees without duplicating transfer data.
Ledger-style transactions	Retains immutable audit history rather than mutating past transfer records.
Fraud log separation	Allows monitoring logic to evolve without contaminating transaction core data.

Figure 4. Database Entity Relationship View

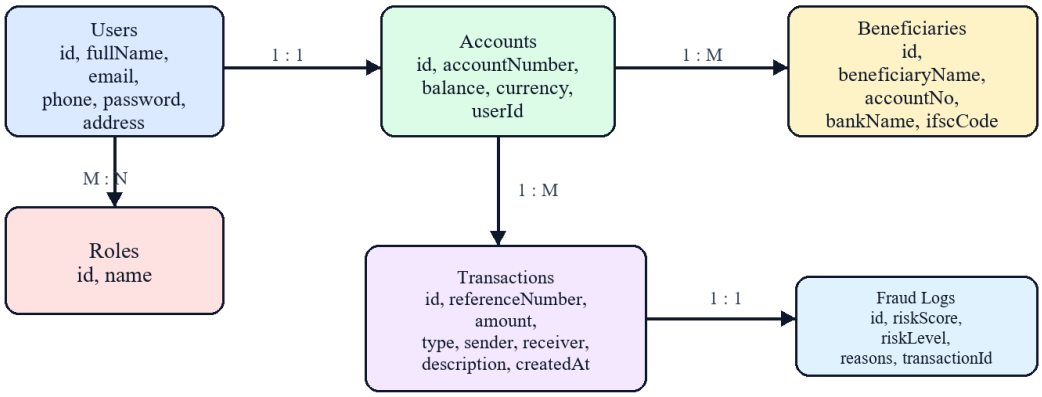

Figure 4. Database Entity Relationship View

7. Authentication and Security

Authentication and authorization are enforced using Spring Security and JWT. Public access is restricted to `/api/auth/**`, Swagger documentation, and API documentation endpoints. Customer routes under `/api/customer/**` are accessible to both customer and admin roles, while admin routes under `/api/admin/**` are restricted to administrators. CSRF is disabled because the application uses stateless token-based authentication rather than server-side form sessions.

Passwords are protected using BCrypt hashing. During login, the backend authenticates the submitted credentials through a `DaoAuthenticationProvider`, and on success it returns a signed JWT token carrying the authenticated identity. The frontend stores this token in local storage and automatically adds it to the `Authorization` header for future API requests using an Axios interceptor.

The `JwtAuthenticationFilter` extends `OncePerRequestFilter` and runs before the standard username-password authentication filter. It extracts the bearer token, resolves the username, loads user details, validates the token, and then populates the Spring Security context. This makes the authenticated principal available to downstream controllers and services without relying on server session state.

Security in NovaBank also extends beyond login. Sensitive entity data such as password hashes are never exposed through API responses because DTOs are used consistently. Transfer execution includes active-account checks, beneficiary ownership validation, and transactional update boundaries. Fraud analysis is stored separately from the transaction ledger to keep monitoring concerns modular and auditable.

Table 9. Security Controls Implemented

Control	Purpose
BCrypt password encoding	Prevents storage of plain-text passwords in the database.
JWT stateless authentication	Enables protected API access without server session dependency.
Role-based route authorization	Restricts admin functions to administrators and protects customer APIs.
DTO-based responses	Avoids exposing entity internals such as password hashes.
Transactional transfer service	Protects money movement with atomic balance updates and consistent ledger writes.
Beneficiary ownership validation	Prevents transfers to unauthorized beneficiary IDs.

Figure 5. Authentication Sequence Diagram

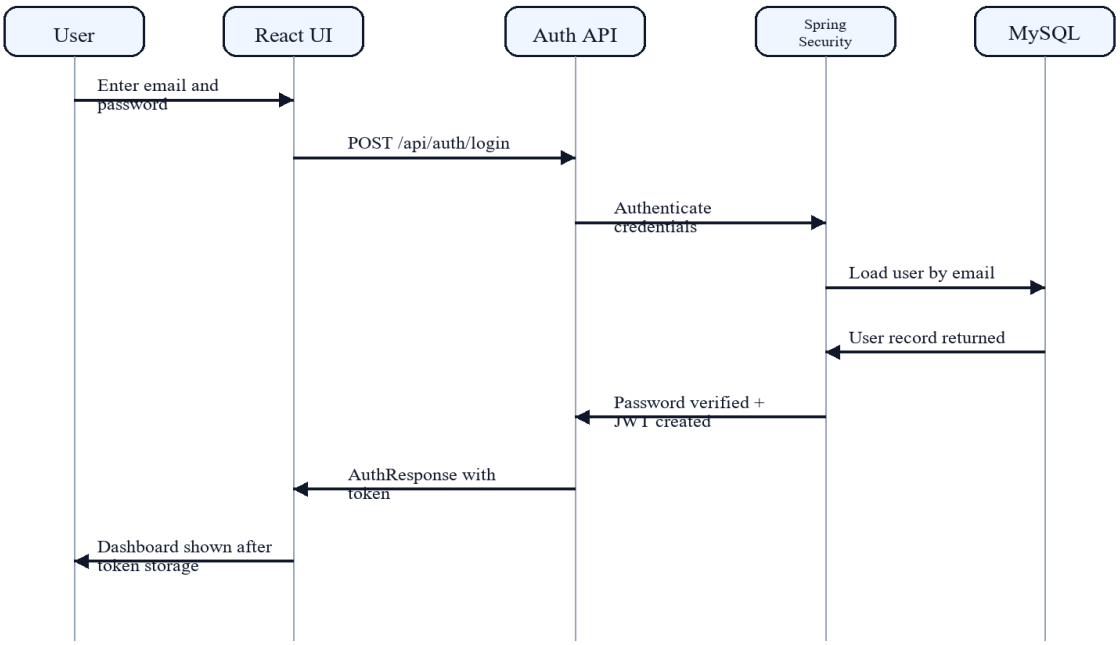

Figure 5. JWT Authentication Sequence

8. Project Management

The project was developed using an iterative and milestone-driven approach similar to Agile practice. Instead of attempting the entire banking system in one step, the work was divided into meaningful phases: requirement analysis, system design, backend foundation, security implementation, customer module development, admin module development, transactional refinement, analytics enhancement, UI redesign, and report preparation.

This approach was useful because banking features are highly interdependent. Authentication had to be stable before customer operations could be protected. Beneficiary management had to be implemented before transfer logic could be meaningfully tested. Transfer logic then had to be corrected and hardened before fraud and insight modules could be evaluated with realistic transaction records. Finally, the frontend was redesigned into a production-style banking interface to improve usability and presentation quality.

Project tracking was centred around deliverable completion rather than formal issue-board tooling. Milestones were validated through API verification, local execution, database inspection, and frontend build checks. Challenges such as duplicate account numbers during seeding, duplicate transaction references, and incomplete receiver-side transfer recording were identified and resolved incrementally during development.

Table 10. Project Milestones

Phase	Milestone	Outcome
Phase 1	Requirements and database planning	Problem definition, user roles, initial schema, and package planning.
Phase 2	Authentication foundation	Registration, login, BCrypt password handling, JWT issue and validation.
Phase 3	Customer service modules	Profile, account summary, beneficiary CRUD, and transaction listing.
Phase 4	Admin operations module	Dashboard metrics, customer review, transaction review, and fraud visibility.
Phase 5	Transfer and analytics	Transactional money transfer, fraud scoring, and financial insight logic.
Phase 6	UI redesign and documentation	Enterprise frontend redesign, build verification, and final report preparation.

Table 11. Implementation Challenges and Resolutions

Challenge	Resolution / Learning
Duplicate seed account numbers	Resolved by making seed logic safer and adjusting account generation strategy.
Duplicate transfer references	Resolved by generating stronger UUID-based references and shared ledger identifiers.
Receiver-side transaction visibility	Resolved by persisting both debit and credit ledger rows for a transfer.
UI realism	Resolved by replacing AI-style glassmorphism with a conservative enterprise banking design system.

9. Results and Evaluation

The final system successfully delivers the core functionality expected from the capstone scope. Customer registration and login are operational, role-based access is enforced, account and profile details are retrievable, beneficiaries can be managed, transactions can be searched, transfers update both sender and receiver ledgers, fraud logs are generated, and financial health analytics are exposed through dedicated APIs and UI pages.

The frontend and backend are integrated end to end. The current frontend build completes successfully, indicating that the refactored interface and routing structure compile into a production bundle. The backend package build also completes successfully, showing that the Java codebase, dependency configuration, and application modules are in a buildable state for deployment and demonstration.

Evaluation of the application is primarily based on functional correctness, architectural soundness, UI clarity, and data integrity rather than on synthetic performance benchmarks. From a usability perspective, the final interface is much more aligned with enterprise banking software than with generic dashboard templates. From a backend perspective, the corrected transfer flow and explicit fraud logging significantly improve domain realism. The main remaining limitations are the absence of a full automated test suite, refresh-token lifecycle management, and production-grade notification workflows.

Table 12. Functional Achievement Matrix

Feature	Status	Evidence
Customer registration and login	Achieved	Public auth APIs and protected customer session flow are operational.
Admin login and admin dashboard	Achieved	Admin routes expose customer, transaction, and fraud review data.
Profile update	Achieved	Customer profile update endpoint and UI page are integrated.
Account balance display	Achieved	Account summary API and dashboard balance widgets are implemented.
Transaction history with search	Achieved	Customer and admin transaction listing support keyword search.
Beneficiary add/edit/delete	Achieved	Beneficiary management is available through protected CRUD endpoints.
Secure money transfer	Achieved	Transfer service updates both sender and receiver ledgers transactionally.

Feature	Status	Evidence
Fraud risk scoring	Achieved	Rule-based scoring and LOW/MEDIUM/HIGH classification are stored in fraud logs.
Financial health score	Achieved	Insight service calculates score, savings rate, trends, and suggestions.
Responsive enterprise UI	Achieved	Frontend refactored into dense banking-style layouts with realistic tables.

Table 13. Evaluation Summary

Dimension	Evaluation
Functional completeness	Core customer and admin flows from the project requirement are implemented end to end.
Security posture	JWT, BCrypt, RBAC, and DTO isolation provide a strong academic security baseline.
Data integrity	Transactional transfer logic and normalized schema improve financial correctness and auditability.
User interface quality	The final frontend resembles an enterprise banking portal rather than a generic dashboard.
Limitations	Automated tests, refresh tokens, and production alerts remain future enhancements.

10. Conclusion

NovaBank demonstrates that a final-year capstone project can be both academically structured and technically realistic when full-stack engineering principles are applied consistently. The project successfully combines frontend design, backend security, relational modelling, transactional banking logic, analytics-oriented modules, and technical documentation in one coherent digital banking system.

The completed application satisfies the original problem statement by providing a secure customer and admin banking platform with protected fund transfer, reusable beneficiary management, searchable transaction history, fraud monitoring, and financial health scoring. The enterprise-style frontend redesign further strengthens the project by making the interface feel like a professional banking product rather than a prototype.

From a learning perspective, the project provided practical experience in Spring Security, JWT authentication, transaction-safe service design, DTO-based API modelling, database normalization, responsive React development, and disciplined report preparation. Overall, NovaBank stands as a strong capstone implementation and a credible demonstration of full-stack engineering capability.

ANNEXURE A. IMPLEMENTED REST APIS

This annexure summarizes the principal API endpoints that support the completed banking workflow.

Method	Endpoint	Purpose
POST	/api/auth/register	Registers a customer and returns authenticated response data.
POST	/api/auth/login	Authenticates a user and returns JWT token with profile details.
GET	/api/customer/profile	Fetches customer profile details.
PUT	/api/customer/profile	Updates profile contact details.
GET	/api/customer/account	Fetches account summary and balance information.
GET	/api/customer/beneficiaries	Lists beneficiaries owned by the logged-in customer.
POST	/api/customer/beneficiaries	Creates a new beneficiary.
PUT	/api/customer/beneficiaries/{id}	Updates an existing beneficiary.
DELETE	/api/customer/beneficiaries/{id}	Removes a beneficiary.
GET	/api/customer/transactions	Lists customer transactions with optional search.
POST	/api/customer/transfer	Executes secured transfer and records sender and receiver ledgers.
GET	/api/customer/fraud-analytics	Returns customer fraud analysis history.
GET	/api/customer/financial-insights	Returns health score, trends, and suggestions.
GET	/api/admin/dashboard	Returns aggregate admin dashboard statistics.
GET	/api/admin/customers	Lists customer details without exposing password data.
GET	/api/admin/transactions	Lists all transactions for administrative review.
GET	/api/admin/fraud-logs	Lists fraud log entries for operations monitoring.

ANNEXURE B. SAMPLE DATA AND PROJECT SNAPSHOT

The following sample records and summary points describe the demo dataset and completed project state used during validation.

User	Email	Role	Address	Account Number
System Admin	admin@bank.com	ROLE_ADMIN	Head Office, Mumbai	--
John Carter	john@bank.com	ROLE_CUSTOMER	Bengaluru, Karnataka	AC100000002
Jane Doe	jane@bank.com	ROLE_CUSTOMER	Hyderabad, Telangana	AC100000003

Project Snapshot

- Frontend routes cover authentication, account overview, transfers, beneficiaries, transaction history, fraud review, financial insights, profile, and admin operations.
- Backend exposes 17 documented REST endpoints across authentication, customer, and admin modules.
- Transfer logic records both sender debit and receiver credit transaction rows under a shared reference.
- Fraud scoring uses practical rule-based conditions and writes outcomes to a dedicated fraud log table.

ANNEXURE C. ADDITIONAL DIAGRAMS

These additional diagrams support explanation of user interaction, transfer flow, and runtime deployment.

Figure 3. Interim Use Case Diagram

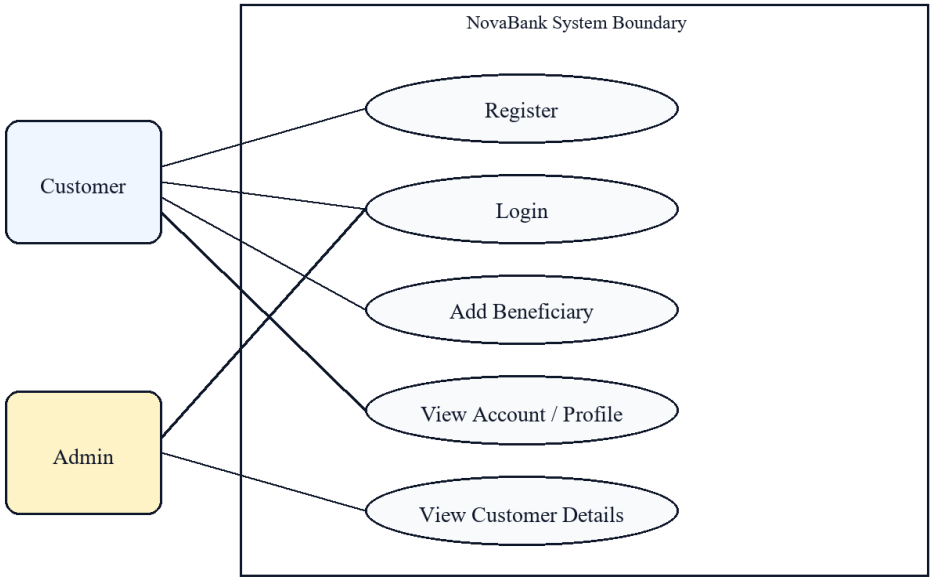

Figure 6. Use Case Diagram

Figure 6. Beneficiary Management Activity Diagram

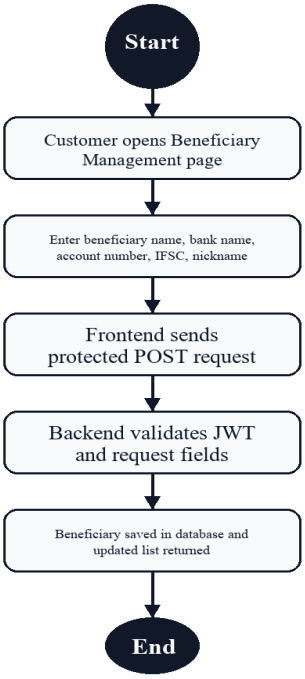

Figure 7. Beneficiary Management Activity Diagram

Figure 7. Deployment / Runtime View

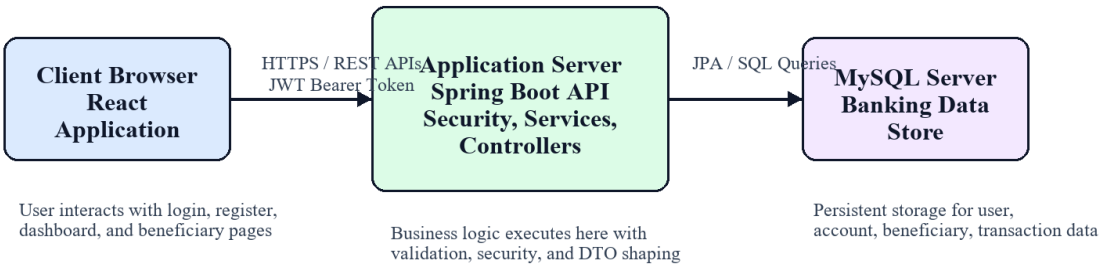

Figure 8. Deployment / Runtime View